Agents

$A$ - Alice
$B$ - Browser with Session storage, SubtleCrypto API, and libsodium library
$M$ - Webauthn authenticator (passkey manager)
$S$ - Quick Crypt server

Browser and Libsodium Functions

$D_H$ - BLAKE2b-512 KDF from libsodium
$D_P$ - PBKDF2-HMAC-SHA512 key derivation FIPS-180-4 from SubtleCrypto
$D_S$ - HMAC-SHA-512 from SubtleCrypto
$E_a$ - Symmetric cipher using AEAD algorithm $a$. One of:
    1. AES-256 in Galois/Counter mode from SubtleCrypto
    2. XChaCha20 with Poly1305 MAC from libsodium
    3. AEGIS 256 from libsodium
$G$ - HMAC-SHA-512 key generator from SubtleCrypto
$H$ - BLAKE2b keyed hash (MAC) generator from libsodium
$H_2$ - SHA-256 from SubtleCrypto
$P_K$ - ML-DSA-65 (FIPS 204) keypair generator from libcrux
$P_S$ - ML-DSA-65 (FIPS 204) randomized signature generator from libcrux
$P_V$ - ML-DSA-65 (FIPS 204) signature verifier from libcrux
$R$ - Cryptographic pseudo-random generator from libsodium
$V$ - Constant-time hash (MAC) validator from libsodium

Cipher Variables

$a$ - Symmetric AEAD cipher and mode: [1, 2, 3]
$ad_F$ - Additional data included in ciphertext
$ad_X$ - $ad_F$ with extended context
$b$ - Valid or invalid MAC tag
$cd$ - Cipher data
$cd_0$ - Cipher data block 0
$cd_N$ - Cipher data block N
$cd_u$ - Encrypted userCred at rest
$err$ - Error message and exit
$f$ - Block flags
$h$ - Password hint text
$h_E$ - Encrypted hint
$h_El$ - Encrypted hint length
$i$ - PBKDF2-HMAC-SHA512 iteration count: min 420,000 max 4,294,000,000
$k_H$ - 256 bit ephemeral hint cipher key
$k_{M0}$ - 256 bit ephemeral message block0 cipher key
$k_{MN}$- 256 bit ephemeral message blockN cipher key
$k_S$ - 256 bit ephemeral MAC key
$k_C$ - 256 bit ephemeral commit key
$kp_B$ - Key purpose text: “Blck_Key”
$kp_C$ - Key purpose text: “Cmit_Key”
$kp_H$ - Key purpose text: “Hint_Key”
$kp_S$ - Key purpose text: “Sign_Key”
$l$ - Payload length
$le$ - Loop end (1-16)
$lp$ - Loop count (1-16)
$m$ - Clear text
$m_0$ - Clear text block 0
$m_N$ - Clear text block N
$m_E$ - Block of encrypted message
$n_{IV}$ - Pseudo random initialization vector
$n_{IV}l$- $n_{IV}$ bit length: [96, 192, 256]
$n_S$ - 128 bit pseudo random salt
$N$ - Block number
$p$ - Password text
$r$ - 384 bits of pseudo random data
$t$ - 256 bit MAC tag
$t_L$ - Last 256 bit MAC tag
$u_c$ - 256 bit user credential in memory
$v$ - Cipher data version: 7

Message Encryption by A

$A \overset{\text{webauthn}}{\leftrightarrow} B, M \overset{\text{webauthn}}{\leftrightarrow} S$
$B$ obtain $u_c$ from $cd_u$ decryption
$A \rightarrow B : m, i, le$
$v = 7$
$lp = 1$
$t_L = \texttt{0x00}$
$LOOP : B$ compute
    $A \rightarrow B : p, h, a$
    $r = R(384)$
    $n_S = r[0 : 128)$
    $n_{IV} = r[128 : 128 + n_{IV}l)$
    $n_{IVH} = D_H(2, kp_H, H(n_S, a, v, lp, n_{IV}))$
    $k_H = D_H(1, kp_H, H(n_S, a, v, lp, u_c))$
    $k_S = D_H(1, kp_S, H(n_S, a, v, lp, u_c))$
    $k_{M0} = D_P(p \parallel u_c \parallel a \parallel v \parallel lp, n_S, i)$
    $k_C = D_H(1, kp_C, H(n_S, k_{M0}))$
    $m_0 \parallel ... \parallel m_N = m$
    $cd = cd_0 \parallel ... \parallel cd_N$
    $m = cd$
    $lp = lp + 1$
    goto LOOP if lp $\leq le$
$A \leftarrow B : cd$

$cd_0$ Encryption by B

$h_E = E_a(h, n_{IVH}, \emptyset, k_H)$
$h_El = len(h_E)$
$f = 1$ if $\boxed{\text{TERM}}$ else 0
$ad_F = f \parallel a \parallel n_{IV} \parallel n_S \parallel i \parallel le \parallel lp \parallel h_El \parallel h_E$
$ad_X = ad_F \parallel k_C$
$m_E = E_a(m_0, n_{IV}, ad_X, k_{M0})$
$l = len(ad_F \parallel m_E)$
$t = H(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S)$
$t_L = t$
$cd_0 = t \parallel v \parallel l \parallel ad_F \parallel m_E$

$cd_N$ Encryption by B

$r = R(384)$
$n_{IV} = r[0 : n_{IV}l)$
$f = 1$ if $\boxed{\text{TERM}}$ else 0
$ad_F = f \parallel a \parallel n_{IV}$
$k_{MN} = D_H(N, kp_B, H(n_S, k_{M0}))$
$ad_X = ad_F \parallel k_C$
$m_E = E_a(m_N, n_{IV}, ad_X, k_{MN})$
$l = len(ad_F \parallel m_E)$
$t = H(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S)$
$t_L = t$
$cd_N = t \parallel v \parallel l \parallel ad_F \parallel m_E$

Message Decryption by A

$A \overset{\text{webauthn}}{\leftrightarrow} B, M \overset{\text{webauthn}}{\leftrightarrow} S$
$B$ obtain $u_c$ from $cd_u$ decryption
$A \rightarrow B : cd$
$t_L = \texttt{0x00}$
$LOOP : B$ compute
    $cd_0 \parallel ... \parallel cd_N = cd$
    $t, v, l, ad_F, m_E = cd_0$
    $f, a, n_{IV}, n_S, i, le, lp, h_E l, h_E = ad_F$
    $k_S = D_H(1, kp_S, H(n_S, a, v, lp, u_c))$
    $b = V(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S, t)$
    $t_L = t$
    if $!b$ :
        $A \leftarrow B : err$
    $n_{IVH} = D_H(2, kp_H, H(n_S, a, v, lp, n_{IV}))$
    $k_H = D_H(1, kp_H, H(n_S, a, v, lp, u_c))$
    $h = E_a^{-1}(h_E, n_{IVH}, \emptyset, k_H)$
    $B \rightarrow A : h$
    $B \leftarrow A : p$
    $k_{M0} = D_P(p \parallel u_c \parallel a \parallel v \parallel lp, n_S, i)$
    $k_C = D_H(1, kp_C, H(n_S, k_{M0}))$
    $m = m_0 \parallel ... \parallel m_N$
    $cd = m$
    goto LOOP if lp $> 1$
$A \leftarrow B : m$

$m_0$ Decryption by B

$ad_X = ad_F \parallel k_C$
$m_0 = E_a^{-1}(m_E, n_{IV}, ad_X, k_{M0})$

$m_N$ Decryption by B

$t, v, l, ad_F, m_E = cd_N$
$f, a, n_{IV} = ad_F$
$b = V(v \parallel l \parallel ad_F \parallel m_E \parallel t_L, k_S, t)$
$t_L = t$
if $!b$ :
    $A \leftarrow B : err$
$k_{MN} = D_H(N, kp_B, H(n_S, k_{M0}))$
$ad_X = ad_F \parallel k_C$
$m_N = E_a^{-1}(m_E, n_{IV}, ad_X, k_{MN})$

$u_c$ Encryption at Rest Variables

$a$ - Symmetric AEAD cipher and mode: 2
$ad_F$ - Additional data included in ciphertext
$ad_X$ - $ad_F$ with extended context
$b$ - Valid or invalid MAC tag
$cd_u$ - Encrypted user credential cipher data
$err$ - Error message and exit
$f$ - Block flags: 1 (fixed)
$h_El$ - Encrypted hint length: 0 (unused)
$i$ - PBKDF2 iteration count: 0 (unused)
$k_C$ - 256 bit ephemeral commit key
$k_{KS}$ - HMAC-SHA-512 non-extractable CryptoKey, stored in IndexedDB
$k_L$ - 256 bit ephemeral per session master key
$k_M$ - 256 bit ephemeral message cipher key
$k_S$ - 256 bit ephemeral MAC key
$kp_C$ - Key purpose text: "Cmit_Key"
$kp_M$ - Key purpose text: "Cphr_Key"
$kp_S$ - Key purpose text: "Sign_Key"
$l$ - Payload length
$le$ - Loop end: 1 (fixed)
$lp$ - Loop count: 1 (fixed)
$m_E$ - Encrypted message
$n_{IV}$ - Pseudo random initialization vector
$n_{IV}l$ - $n_{IV}$ bit length: 192
$n_S$ - 128 bit pseudo random salt
$pkId$ - Passkey credential identifier
$r$ - 320 bits of pseudo random data
$s$ - IndexedDB slot label: "user-cred-key"
$t$ - 256 bit MAC tag
$u_c$ - 256 bit user credential (plaintext)
$u_i$ - User id
$v$ - Cipher data version: 7

$k_L$ Derivation by B

$B \leftarrow M : pkId$
$k_{KS} = G()$
$k_L = D_S(s \parallel pkId, k_{KS})[32:64]$

$u_c$ Encryption by B

$B \leftarrow S : u_c, u_i$
$r = R(320)$
$n_S = r[0:128)$
$n_{IV} = r[128:128 + n_{IV}l)$
$k_M = D_H(0, kp_M, H(n_S, a, v, lp, u_i, k_L))$
$k_S = D_H(1, kp_S, H(n_S, a, v, lp, u_i, k_L))$
$k_C = D_H(1, kp_C, H(n_S, k_M))$
$ad_F = f \parallel a \parallel n_{IV} \parallel n_S \parallel i \parallel le \parallel lp \parallel h_El$
$ad_X = ad_F \parallel u_i \parallel k_C$
$m_E = E_a(u_c, n_{IV}, ad_X, k_M)$
$l = len(ad_F \parallel m_E)$
$t = H(v \parallel l \parallel ad_F \parallel m_E \parallel \texttt{0x00}, k_S)$
$cd_u = t \parallel v \parallel l \parallel ad_F \parallel m_E$

$cd_u$ Decryption by B

$B \leftarrow S : u_i$
$B \leftarrow$ session storage : $cd_u$
$t, v, l, ad_F, m_E = cd_u$
$f, a, n_{IV}, n_S, i, le, lp, h_E l = ad_F$
$k_S = D_H(1, kp_S, H(n_S, a, v, lp, u_i, k_L))$
$b = V(v \parallel l \parallel ad_F \parallel m_E \parallel \texttt{0x00}, k_S, t)$
if $!b$ :
    $A \leftarrow B : err$
$k_M = D_H(0, kp_M, H(n_S, a, v, lp, u_i, k_L))$
$k_C = D_H(1, kp_C, H(n_S, k_M))$
$ad_X = ad_F \parallel u_i \parallel k_C$
$u_c = E_a^{-1}(m_E, n_{IV}, ad_X, k_M)$

Authentication Variables

$ao$ - Authentication options, including $o, ch$
$ar$ - Authentication response, including signed $ch$
$bd$ - HTTP request body
$cd_u$ - Encrypted userCred at rest
$ch$ - 256 bit challenge value
$ch_R$ - 256 bit recovery challenge value
$ck$ - Session cookie, a signed JWT
$cs$ - CSRF token
$cw$ - Alice's webauthn authenticator credentials
$kp_R$ - Key purpose text: "RecovKey"
$kp_U$ - Key purpose text: "UCredKey"
$m_R$ - Recovery proof message
$m_U$ - userCred proof message
$mth$ - HTTP request method
$n_P$ - 256 bit proof nonce value
$o$ - Quick Crypt origin "https://quickcrypt.org"
$pk_R$ - ML-DSA-65 recovery proof public key
$pk_U$ - ML-DSA-65 userCred proof public key
$pth$ - HTTP request path
$ro$ - Registration options, including $o, ch, u_i$
$rq$ - Authorized request, including $mth, pth, bd$
$rr$ - Registration response, including signed $ch$
$sc_R$ - Recovery signature context: "qcrypt/recovery/proof/v1"
$sc_U$ - userCred signature context: "qcrypt/usercred/proof/v1"
$sk_R$ - ML-DSA-65 recovery proof secret key
$sk_U$ - ML-DSA-65 userCred proof secret key
$ts$ - Request timestamp in milliseconds
$u_c$ - 256 bit user credential in memory
$u_i$ - 128 bit user id guaranteed to be unique
$u_n$ - A's chosen user name
$u_r$ - 128 bit recovery id
$\Delta$ - Maximum request timestamp skew: 60 seconds
$\delta$ - Delimiter byte: \n
$\sigma_R$ - ML-DSA-65 recovery proof signature
$\sigma_U$ - ML-DSA-65 userCred proof signature

Registration by A

$A \to B : u_n$
$B \to S : u_n, o$
$S : u_i = R(128)$
$S : ch = R(256)$
$S$ store $u_i, ch$
$B \leftarrow S : ro$
$B \to M : ro$
$A \to M : cw$
$M$ create and store passkey, sign $ch$
$B \leftarrow M : rr$
$B : u_r = R(128)$
$B : (pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
$B \to S : rr, u_i, ch, pk_R$
$S : u_c = R(256)$
$S \to KMS : cd_u$encrypt $uc$ with KMS
$S : (pk_U, sk_U) = P_K(D_H(1, kp_U, u_c))$
$S$ verify webauthn signature, store $rr, pk_R, pk_U$ and $cd_u$, remove $ch$
$S$ create $ck, cs$
$B \leftarrow S : u_i, u_n, u_c, cs, ck$
$B$ store $cd_u$ from $u_c$ encryption
$A \leftarrow B : u_r \parallel u_i$ as BIP39

Authentication by A

$B \to S : o[, u_i]$
$S : ch = R(256)$
$S$ store $ch$
$B \leftarrow S : ao$
$B \to M : ao$
$A \to M : cw$
$M$ sign $ch$
$B \leftarrow M : ar$
$B \to S : ar, ch$
$S$ verify webauthn signature, remove $ch$
$S$ create $ck, cs$
$B \leftarrow S : u_i, u_n, u_c, cs, ck$
$B$ store $cd_u$ from $u_c$ encryption

Authorization of B

$B$ obtain $u_c$ from $cd_u$ decryption
$B : (pk_U, sk_U) = P_K(D_H(1, kp_U, u_c))$
$B : ts =$ current time
$B : n_P = R(256)$
$B : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts\delta \parallel n_P\delta \parallel H_2(bd)$
$B : \sigma_U = P_S(sk_U, m_U, sc_U)$
$B \to S : rq, ck, cs, ts, n_P, \sigma_U$
$S$ verify $ck, cs$
$S$ verify $ts$ within $\Delta$
$S : m_U = u_i\delta \parallel mth\delta \parallel pth\delta \parallel ts\delta \parallel n_P\delta \parallel H_2(bd)$
$S : b = P_V(pk_U, m_U, \sigma_U, sc_U)$
if $mth$ is state-changing:
    $S : b$ &= $n_P$ is unique
if !$b$ :
    $B \leftarrow S : err$
    $A \leftarrow B : err$
$B \leftarrow S :$ rq response

Recovery from Lost Passkey by A

$A \rightarrow B : u_r \parallel u_i$ as BIP39
$B \rightarrow S : u_i, o$
$S : ch_R = R(256)$
$S$ store $ch_R$
$B \leftarrow S : ch_R$
$B : (pk_R, sk_R) = P_K(D_H(1, kp_R, u_r \parallel u_i))$
$B : m_R = u_i\delta \parallel ch_R$
$B : \sigma_R = P_S(sk_R, m_R, sc_R)$
$B \rightarrow S : u_i, ch_R, \sigma_R$
$S$ remove $ch_R$
$S : m_R = u_i\delta \parallel ch_R$
$S : b = P_V(pk_R, m_R, \sigma_R, sc_R)$
if $!b$ :
    $B \leftarrow S : err$
    $A \leftarrow B : err$
$S : ch = R(256)$
$S$ delete existing $rr$, store $ch$
$B \leftarrow S : ro$
$B \rightarrow M : ro$
$A \rightarrow M : cw$
$M$ create and store passkey, sign $ch$
$B \leftarrow M : rr$
$B \rightarrow S : rr, u_i, ch$
$S$ verify webauthn signature, store $rr$, remove $ch$
$S$ create $ck, cs$
$B \leftarrow S : u_i, u_n, u_c, cs, ck$
$B$ store $cd_u$ from $u_c$ encryption